

Automatic Installation Of Docker and Kubernetes Cluster

Tasks :

Install Ansible server

1. Introduction
2. System requirements for Ansible
3. Install Ansible server
4. Setup SSH keys and share it among managed nodes
5. Setting up the Inventory File for docker servers
6. Testing the Connection

Proxy setting (environment and apt)

7. Proxy setting for servers in environment and apt

Install and config Docker

8. Install docker with Ansible on servers
9. Config Cgroup with systemd

Proxy setting (Docker)

10. Proxy setting for docker servers

Get status and docker login

11. Get status of docker service
12. Automatic docker login on servers
13. Test image pull of docker hub

Install and config kubernetes cluster

14. Setting up the Inventory File for kubernetes cluster
15. Install kubernetes with Ansible on servers
16. Install and configure dependencies
17. Creating a Master Node using Ansible Playbook
18. Join Worker Nodes to Kubernetes Cluster using Ansible Playbook
19. Testing the status of the cluster nodes

Install and config Metrics server

20. Install metrics server
21. Config metrics server
22. Testing the status of the metrics server

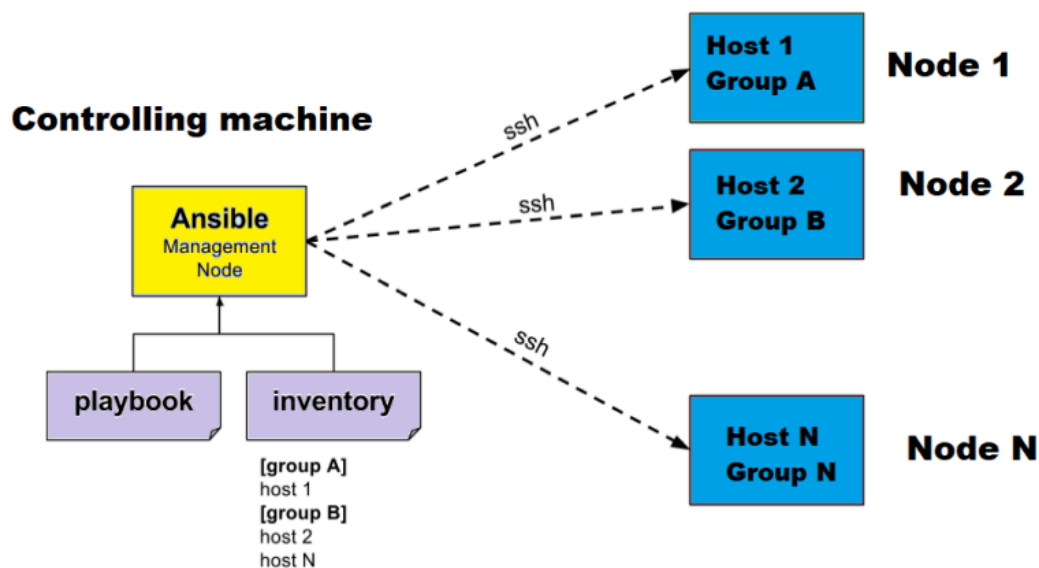
STEP 1

Introduction

Ansible is an IT automation tool. It can configure systems, deploy software, and orchestrate more advanced IT tasks such as continuous deployments or zero downtime rolling updates.

Ansible's main goals are simplicity and ease-of-use. It also has a strong focus on security and reliability, featuring a minimum of moving parts, usage of OpenSSH for transport (with other transports and pull modes as alternatives), and a language that is designed around auditability by humans—even those not familiar with the program.

We believe simplicity is relevant to all sizes of environments, so we design for busy users of all types: developers, sysadmins, release engineers, IT managers, and everyone in between. Ansible is appropriate for managing all environments, from small setups with a handful of instances to enterprise environments with many thousands of instances.



STEP 2

System requirements for Ansible

Minimal Installed Ubuntu 20.04 LTS / 21.04

sudo user with root privileges

2 CPU / vCPU

2 GB RAM or more

20 GB Hard drive

STEP 3

Install Ansible server

Node : ansible server

Apply Updates on ansible server :

`sudo apt update`

Install ansible dependencies :

`sudo apt install -y software-properties-common`

Now update repository :

`sudo apt update`

Install latest version of ansible:

`sudo apt install -y ansible`

After the successful installation of Ansible, verify its version by executing the command :

`ansible --version`

STEP 4

Setup SSH keys and share it among managed nodes

Node : ansible server

Enter the following command in the Ansible control node terminal:

ssh-keygen

Hitting Enter shows the following output :

```
test@test-machine:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/test/.ssh/id_rsa):
```

The output should be similar to the one below :

```
Your identification has been saved in /home/test/.ssh/id_rsa
Your public key has been saved in /home/test/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:GKUPN9UmR4WDrabMe3fi9AmCfyNFqjq5vXne1aBZoIw test@test-machine
The key's randomart image is:
+---[RSA 3072]-----+
|  .  .=.o.  |
|  o .o B    |
| + o = .    |
| * .o .     |
| .oSo o. o   |
|  +o...= o  |
| .Eo+.= . . |
| o.o++++=+.. |
| o++=o=o+o   |
+---[SHA256]-----+
```

Ansible hosts are remote servers that are monitored and controlled by the Ansible control node. Each host needs to have the control node's SSH key in the system user's `authorized_keys` directory.

setting up an SSH public key to copy it using the `ssh-copy-id` command:

ssh-copy-id username@remote_host

```
test@test-machine:~$ ssh-copy-id test@10.0.2.15
The authenticity of host '10.0.2.15 (10.0.2.15)' can't be established.
ECDSA key fingerprint is SHA256:ecAnmW4+iBei5bdFM2ekWPsjC5//L+DDYj9o6Q7M0c.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

Server : a-kube-master

ssh-copy-id a-kube-master@x.x.x.x

password : 123

Server : a-kube-w1

ssh-copy-id a-kube-w1@x.x.x.x

password : 123

Server : a-kube-w2

ssh-copy-id a-kube-w2@x.x.x.x

password : 123

Server : ansible-server

ssh-copy-id ansible-server@x.x.x.x

password : 123

STEP 5

Setting up the Inventory File

Node : ansible-server

To access the inventory file, use the following command in the control node's terminal:

The default path for the Ansible inventory file is **/etc/ansible/hosts**

```
sudo nano /etc/ansible/hosts
```

Scroll to the bottom of the file to add new hosts and sort them into categories.

[category name]

[server_name] ansible_host=[server_ip]

After you've set up the inventory file, you can always check it again by using :

```
ansible-inventory --list -y
```

STEP 6

Testing the Connection

Node : ansible server

To test the connection with the hosts, use the following command :

```
ansible all -m ping
```

Running Ad-Hoc Commands :

Get Status of hard disks:

```
ansible all -a "df -h" -u root
```

Get Status of uptimes:

```
ansible all -a "uptime" -u root
```

STEP 7

Proxy setting for servers in environment and apt

Node : ansible server

Create directory for setting proxy :

```
mkdir ansible-proxy
```

```
cd ansible-proxy
```

Create Ansible playbook to set proxy :

```
vim set-proxy.yml
```

copy configuration of set-proxy.yml :



set-proxy.yml

Install and config Docker

STEP 8

Install docker with Ansible on servers

Node : ansible server

Create directory for playbooks

```
cd
```

```
mkdir ansible-docker
```

```
cd ansible-docker
```

create a file named docker.yml

```
vim docker.yml
```

copy configuration of docker.yml :



docker.yml

STEP 9

Config Cgroup with systemd

Node : ansible server

create a file named daemon.json

```
vim daemon.json
```

copy configuration of daemon.json :



daemon.yml

Now run the playbook using ansible-playbook command :

```
ansible-playbook docker.yml --ask-become-pass
```

```
password : x
```

STEP 10

Proxy setting for docker servers

Node : ansible server

Create Ansible playbook to set docker proxy:

```
nano docker-proxy.yml
```

copy configuration of docker-proxy.yml :



docker-proxy.yml

Now run the playbook using ansible-playbook command :

```
ansible-playbook docker-proxy.yml --ask-become-pass
```

```
password : x
```

STEP 11

Get status of docker service

Node : ansible server

Running Ad-Hoc Commands

Get Status of docker services :

```
ansible docker-servers -a "systemctl status docker"
```

For every 3 nodes we have to get status = active (running)

STEP 12

Automatic docker login on servers

Node : ansible server

Create Ansible playbook to set docker proxy:

```
vim docker-login.yml
```

copy configuration of docker-login.yml :



docker-login.yml

Now run the playbook using ansible-playbook command :

```
ansible-playbook docker-login.yml --ask-become-pass
```

password : x

STEP 13

Test image pull of dockerhub

Node : ansible server

Running Ad-Hoc Commands

Get Status of docker services :

```
sudo usermod -aG docker ${USER}
```

```
ansible docker-servers -a "docker pull hello-world" -b --ask-become-pass
```

password : x

Install and config kubernetes cluster

STEP 14

Setting up the Inventory File for deploy kubernetes

Node : ansible server

To access the inventory file, use the following command in the control node's terminal:

The default path for the Ansible inventory file is **/etc/ansible/hosts**

```
sudo nano /etc/ansible/hosts
```

Scroll to the bottom of the file to add new hosts and sort them into categories.

[category name]

```
server_name ansible_host=[server_ip]
```

add servers :

[kubernetes-servers]

```
a-kube-master ansible_host=x.x.x.x ansible_user=a-kube-master  
ansible_pass=x
```

```
a-kube-worker1 ansible_host=x.x.x.x ansible_user=a-kube-w1 ansible_pass=x
```

```
a-kube-worker2 ansible_host=x.x.x.x ansible_user=a-kube-w2 ansible_pass=x
```

After you've set up the inventory file, you can always check it again by using :

```
ansible-inventory --list -y
```

STEP 15

Install kubernetes with Ansible on servers

Node : ansible server

Create directory for playbooks

```
cd
```

```
mkdir ansible-kube
```

```
cd ansible-kube
```

STEP 16

Install and configure dependencies

Node : ansible server

create a file named kube-dependencies.yml

```
nano kube-dependencies.yml
```

copy configuration of kube-dependencies.yml :



kube-dependencies
.yml

Now run the playbook using ansible-playbook command :

```
ansible-playbook kube-dependencies.yml --ask-become-pass
```

password : x

STEP 17

Creating a Master Node using Ansible Playbook

Node : ansible server

create a file named master-cluster.yml

```
nano master-cluster.yml
```

copy configuration of master-cluster.yml :



master-cluster.yml

Install pod network :

create a file named kube-flannel.yml

```
nano kube-flannel.yml
```

copy configuration of kube-flannel.yml :



kube-flannel.yml

Now run the playbook using ansible-playbook command :

```
ansible-playbook master-cluster.yml --ask-become-pass
```

password : x

STEP 18

Join Worker Nodes to Kubernetes Cluster using Ansible Playbook

Node : ansible server

create a file named workers-cluster.yml

```
vim workers-cluster.yml
```

copy configuration of workers-cluster.yml :



workers-cluster.yml

Now run the playbook using ansible-playbook command :

```
ansible-playbook workers-cluster.yml --ask-become-pass
```

password : x

STEP 19

Testing the status of the cluster Nodes and Pods

Node : ansible server

Running Ad-Hoc Commands :

```
ansible a-kube-master -a "kubectl get pods -o wide -n kube-system" -u root
```

status = Running

```
ansible a-kube-master -a "kubectl get nodes" -u root
```

status = Ready

STEP 20

Install metrics server

Node : ansible server

create a file named metrics-server.yml

```
vim metrics-server.yml
```

copy configuration of metrics-server.yml :



metrics-server.yml

STEP 21

Config and apply metrics server

Node : ansible server

create a file named components.yml

```
nano components.yml
```

copy configuration of components.yml :



components.yml

Now run the playbook using ansible-playbook command :

```
ansible-playbook metrics-server.yml --ask-become-pass
```

password : x

STEP 22

Testing the status of the metrics server

Node : ansible server

Running Ad-Hoc Commands :

Check metrics server :

```
ansible a-kube-master -a "kubectl get pods -n kube-system" -u root  
metrics server pod status must be running
```

```
ansible a-kube-master -a "kubectl top nodes" -u root
```

Columns related to CPU and memory have a value

Successful

END